

Flink 原理详解 - 第 15 讲

生产部署、HA、升级与上线检查：从“能跑”到“能上线”

前 14 讲已经把流处理本身的计算逻辑讲通。这一讲换一个视角：同样的 Job，如果放到真实生产环境中，怎样保证它能长期运行、机器坏了能恢复、版本升级能回滚、问题出现后能定位。

本讲目标：建立生产级最低闭环：会选择部署方式，会配置 HA/Checkpoint，会用 Savepoint 升级，会区分故障层次，会做上线验收。

上线前：最低生产闭环检查单

正确性	容错	性能	运维
☐ Event Time/Watermark	☐ Checkpoint	☐ 并行度	☐ 监控告警
☐ State/TTL	☐ HA	☐ 反压	☐ 日志
☐ Sink 语义	☐ Savepoint	☐ State 大小	☐ 升级回滚
☐ 手工基准结果	☐ 重启策略	☐ Sink 吞吐	☐ 值班手册

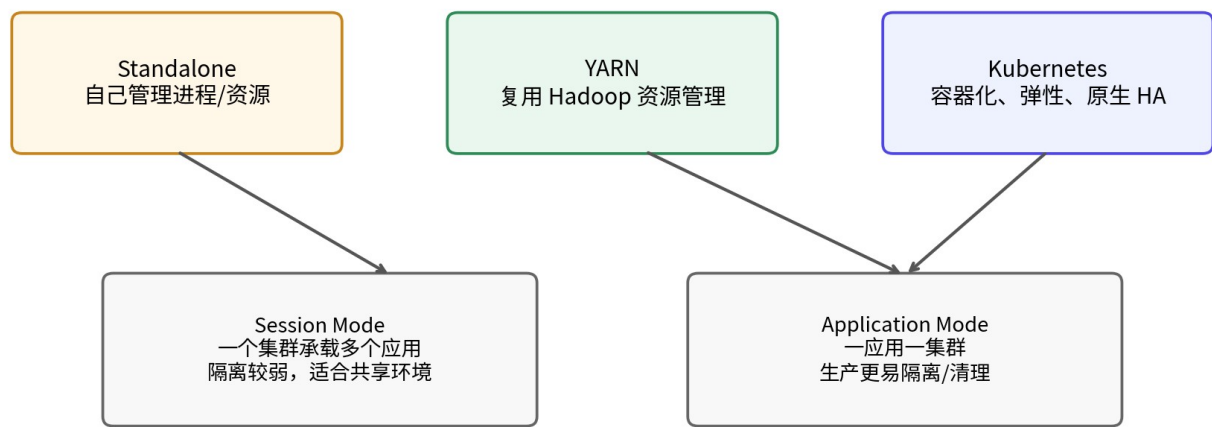
上线不是“部署成功”，而是：能验证、能观察、能恢复、能升级、能回滚。

学习结束后应能回答	答案关键词
Standalone、YARN、Kubernetes 怎么选？	看资源管理、隔离、弹性和团队基础设施
Checkpoint 与 Savepoint 谁负责什么？	自动故障恢复 vs 主动升级/迁移
JobManager 挂了为什么还能恢复？	HA 选主 + 外部持久化元数据/状态
升级怎样尽量不丢数据？	Savepoint → 停旧 → 起新 → 验证 → 可回滚
上线成功的定义是什么？	正确 + 可观测 + 可恢复 + 可升级

1. 生产部署不是“换一条启动命令”

开发环境关注“代码能不能算对”；生产环境多了四类系统问题：资源从哪里来、组件怎么恢复、状态放哪里、版本怎么升级。

Flink 生产部署：先决定“集群怎么活”



生产环境真正要权衡的是：资源隔离、运维复杂度、弹性、HA 与团队现有基础设施。

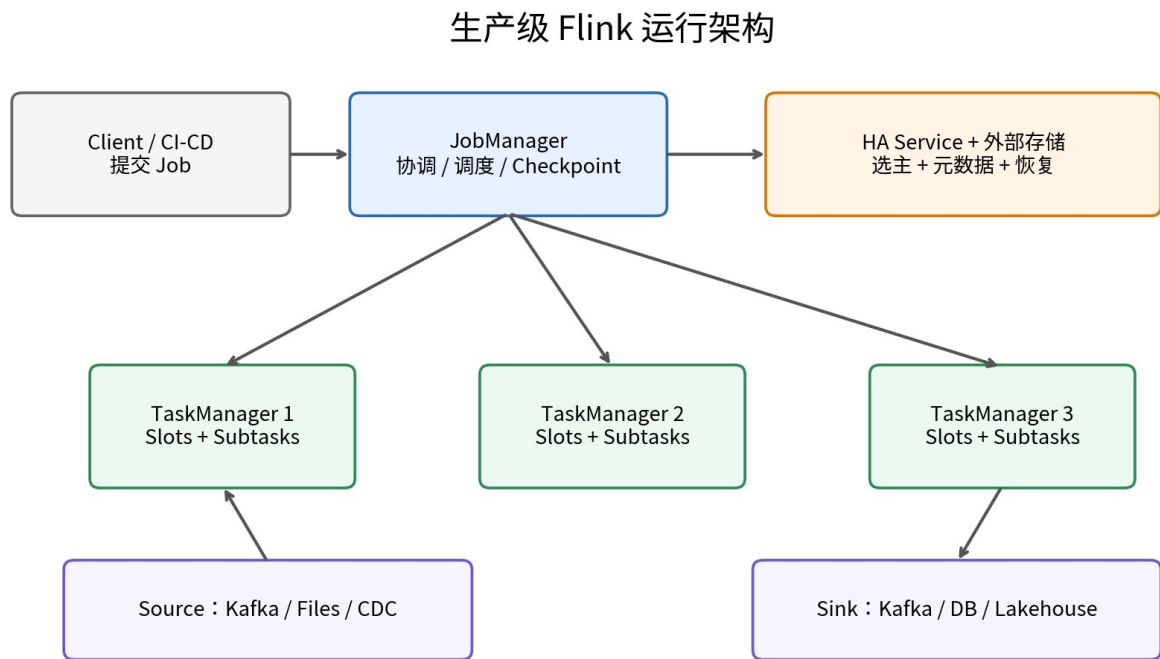
方式	适合场景	优势	主要代价
Standalone	教学、固定机器、小规模独立集群	结构直接、易理解	资源伸缩和 HA 运维更多靠自己
YARN	已有 Hadoop/YARN 的组织	复用现有资源池和权限体系	与 YARN 生命周期/队列配置绑定
Native Kubernetes	容器化、弹性、云原生生产环境	隔离、弹性、自动化能力强	K8s 运维门槛更高

对你当前学习环境：已有 Hadoop 3.3.5/YARN 时，先理解和练习 YARN/Standalone 最划算；Kubernetes 先掌握架构，不必为了学习 Flink 立即搭一套 K8s。

Session Mode 与 Application Mode

Session Mode 是“一个长期集群承载多个应用”；Application Mode 更接近“一应用一集群”，应用 main() 在集群侧执行，资源和故障隔离更清楚。官方当前文档对生产 Kubernetes 场景明确更推荐 Application Mode。

2. 一张图看懂生产级 Flink 架构



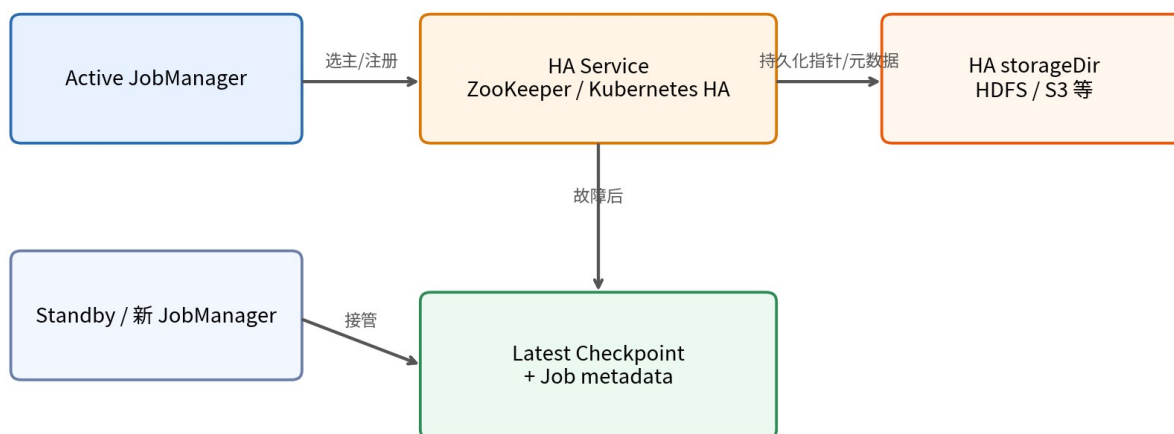
真正的生产系统通常至少包含：提交端、JobManager、多个 TaskManager、外部 HA 服务、外部 Checkpoint/Savepoint 存储，以及真正的 Source/Sink。

组件	生产职责	不能只看什么
Client / CI-CD	打包、提交、参数化、版本记录	不能只靠手工命令历史
JobManager	调度、Checkpoint 协调、故障恢复	不能把它当成唯一状态保存点
TaskManager	执行 Subtask、持有运行时状态	单台宕机应可重建
HA Service	选主、JobManager 故障接管	本机文件不够
外部存储	Checkpoint/HA 元数据/Savepoint	必须高可用、可持久
Source/Sink	决定端到端语义与吞吐上限	Flink 内部正确不代表外部一定正确

底层原则：计算进程可以死，状态和恢复信息不能只活在计算进程本机。

3. 高可用 HA : JobManager 挂了以后发生什么？

HA：不是“永远不失败”，而是“失败后快速接管”



核心：JobManager 可替换；真正不能只存在本机的是恢复所需元数据和状态快照。

HA 不是让 JobManager 永远不挂，而是把“谁是 Leader”和“恢复所需元数据”放到可接管的机制中。Kubernetes HA 会借助 Kubernetes 资源完成选主，同时把恢复元数据持久到 high-availability.storageDir 指向的外部文件系统。

故障恢复需要两类东西

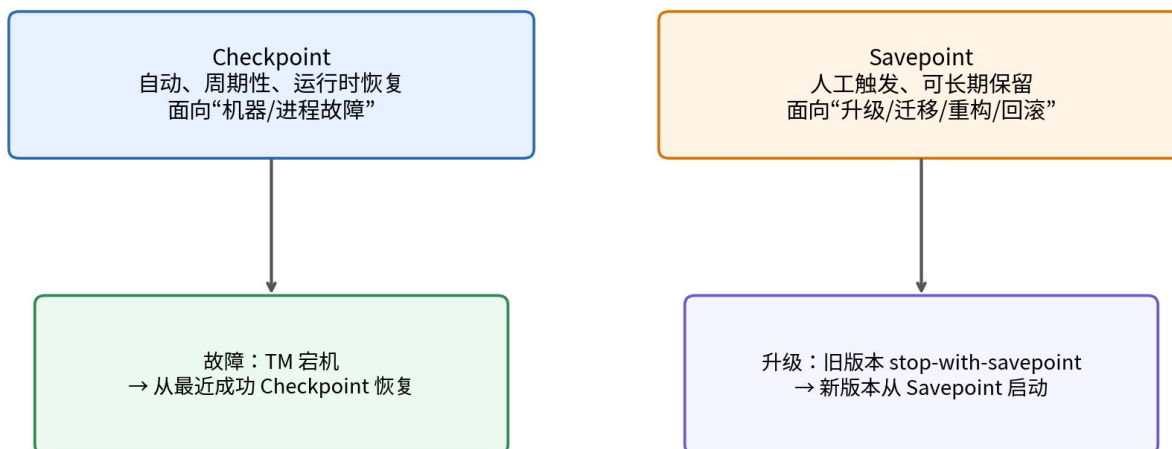
- 控制面元数据：JobGraph、Leader/Job 状态等；
- 数据面状态：最近成功的 Checkpoint 中的 Operator/Keyed State、Source 位置等。

```
# Kubernetes HA 的典型思路（示意）
high-availability.type: kubernetes
high-availability.storageDir: hdfs:///flink/recovery
kubernetes.cluster-id: realtime-orders
```

常见误区：“开了 Checkpoint = JobManager HA” 不成立。Checkpoint 解决作业状态恢复；JobManager HA 解决协调组件本身的故障接管，两者互补。

4. Checkpoint 与 Savepoint : 两个快照，两个目的

Checkpoint 与 Savepoint : 自动恢复 vs 主动运维



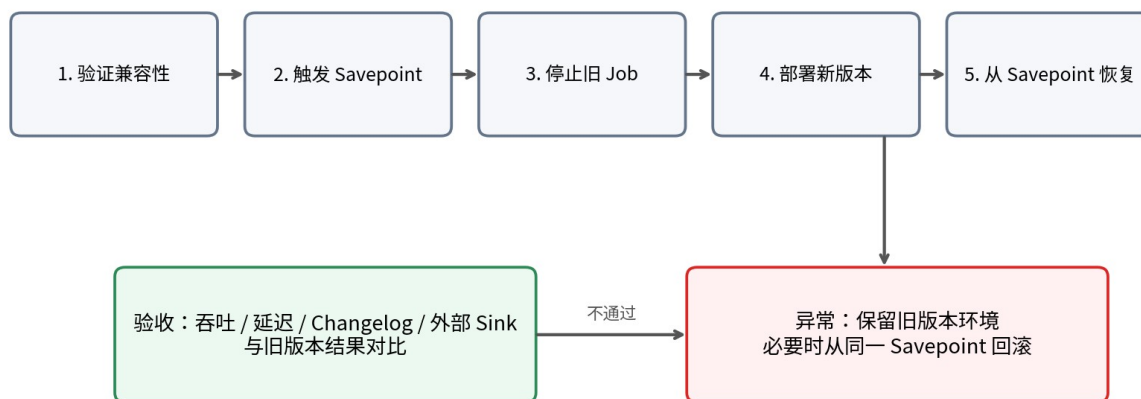
对比项	Checkpoint	Savepoint
主要目的	自动故障恢复	人工运维、升级、迁移、回滚
触发方式	周期性自动触发	CLI/REST/运维流程主动触发
生命周期	运行时管理，可被清理	通常由运维人员明确保存
典型场景	TM 崩溃后自动恢复	升级 Jar、改并行度、迁移集群
心智模型	“自动存档点”	“人为打的版本存档”

升级为什么更依赖 Savepoint ?

因为升级是“有计划地改变程序”。你需要一个明确、可命名、可保留的状态边界，并让新版本从这个边界恢复，而不是依赖运行时可能被清理的普通 Checkpoint。

5. 升级、扩缩容与回滚：别直接杀进程换 Jar

生产升级的安全闭环



官方升级文档的核心思想非常稳定：先在旧版本创建一致 Savepoint，再在新版本从这个 Savepoint 恢复。升级可能是 Flink 版本变化，也可能只是业务代码、配置或并行度变化。

```
# 1) 生成 Savepoint (示意)
./bin/flink savepoint <jobId> hdfs:///flink/savepoints

# 也可以 stop-with-savepoint，让作业在一致点优雅停止
./bin/flink stop --savepointPath hdfs:///flink/savepoints <jobId>

# 2) 新版本从 Savepoint 启动
./bin/flink run -s hdfs:///flink/savepoints/savepoint-xxx new-job.jar
```

为什么 operator UID 重要？

有状态作业升级时，Flink 要把 Savepoint 里的状态重新映射到新 JobGraph 中对应的算子。稳定的 Operator UID 能显著降低“代码只是重排了一下，状态却找不到”的风险。

上线建议：任何有状态作业，在第一次进入生产前就给关键有状态 Operator 设计稳定 UID，不要等第一次升级时才补。

6. Restart Strategy : 失败后“要不要重试、怎么重试”

HA 和 Checkpoint 决定 “能不能恢复” ； Restart Strategy 决定 “失败后什么时候、重试几次、采用什么节奏恢复” 。

策略思路	适合情况	风险
fixed-delay	偶发故障、依赖服务短暂抖动	间隔太短可能反复撞同一故障
failure-rate	允许一定时间窗口内有限次数失败	参数需要结合业务故障模式
exponential-delay	外部依赖恢复时间不确定	恢复可能更慢但减少重试风暴

```
# SQL/TableConfig 中的配置示例（官方文档）
restart-strategy.type: fixed-delay
restart-strategy.fixed-delay.attempts: 3
restart-strategy.fixed-delay.delay: 30s

execution.checkpointing.mode: EXACTLY_ONCE
execution.checkpointing.interval: 3min
```

原则：Restart Strategy 不能把永久性代码错误变成 “自动修复” 。如果反复同样失败，继续重启只是在制造噪声。

7. 故障诊断：先问“哪一层坏了？”

故障不是一种：不同故障看不同证据

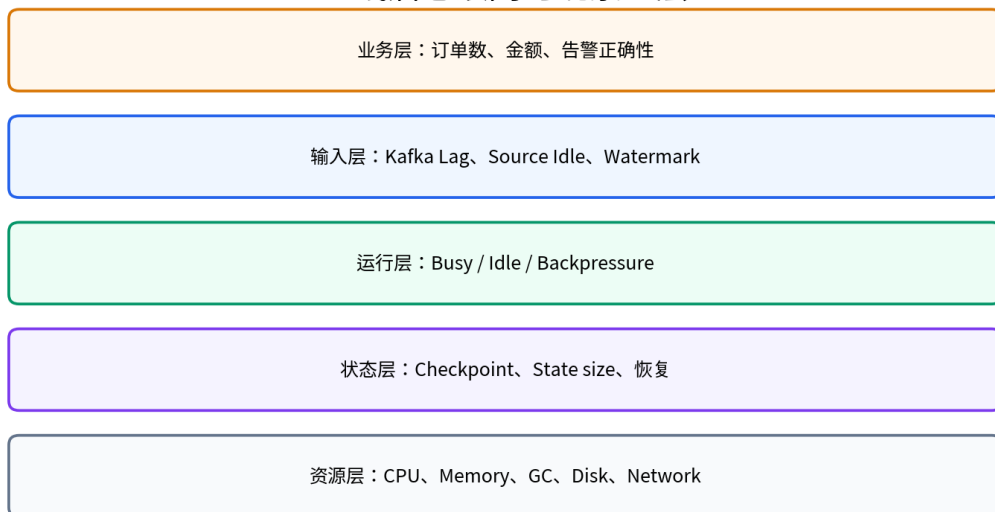


现象	第一批证据	不要先做
JobManager 反复重启	HA/Leader 日志、外部存储可达性	先调并行度
某 TaskManager 消失	TM 日志、容器退出码、OOM/GC	先改 Watermark
Kafka Lag 上升	Source Lag + Backpressure 链	先扩 Kafka 分区却不看下游
Sink HIGH backpressure	Sink busy、外部 DB/Kafka 延迟	盲目加 Source 并行度
Checkpoint 变慢	start delay/alignment/size/async	只把 interval 调大
升级无法从 Savepoint 恢复	UID、state schema、拓扑兼容性	直接删 Savepoint

诊断方法：把故障当成“分层系统”问题：资源层、运行时层、状态层、Source/Sink 层、业务正确性层。

8. 控：Job 显示 RUNNING 远远不够

上线后必须同时观察 5 层



只看“Job RUNNING”远远不够：生产可用性 = 正确性 + 可恢复性 + 可观察性。

至少要覆盖五类指标

- 业务正确性：订单数、金额、告警数、重复/漏数；
- 输入：Kafka Lag、Source Idle、Watermark Lag；
- 运行：Busy/Idle/Backpressure、records in/out；
- 状态：Checkpoint 成功率、大小、时延、State 大小；
- 资源：CPU、JVM/Managed Memory、GC、磁盘、网络。

生产告警应尽量基于“用户能感知的问题”与“系统早期信号”组合。例如：Kafka Lag 持续增长 + Sink Backpressure 高，比单独“CPU 80%”更具有业务解释力。

日志保留也要设计

至少能关联：Job 版本、JobID、部署时间、Savepoint、配置快照、异常堆栈。否则问题发生两天后，你可能连“当时跑的是哪一版 Jar”都无法确认。

9. 资源与部署模式：不要把生产参数当公式

问题	判断方式
TaskManager 要多少？	由目标吞吐、并行度、slot、每个 Subtask 的 CPU/内存实际使用推导
每个 TM 多大内存？	看 state backend、managed memory、heap、network buffer、GC
并行度多大？	看 Source partition、热点 Key、最慢 Operator、Sink 能力
Session 还是 Application？	看隔离、生命周期、提交成本、生产稳定性需求
YARN 还是 K8s？	看现有基础设施与运维能力，不是看“谁更新”

资源调优原则：先在稳定负载下测量单 Subtask 可持续吞吐，再反推并行度；不要从“网上推荐 executor/taskmanager 参数”倒推系统。

版本环境提醒

Flink 2.x 的 Java 要求与老 Hadoop 教学环境不同。学习/生产迁移时更稳妥的做法是让 JDK 8 与 JDK 17 并存，按组件显式切换 JAVA_HOME，而不是直接替换 Hadoop 正在使用的 JDK。

10. 一个生产升级案例：实时订单聚合 Job

背景：Kafka orders_raw → Flink 窗口聚合 → upsert-kafka / 数据库。当前 Job 已运行三个月，现在要把城市维度改成“城市 + 渠道”，并把并行度从 4 调到 8。

步骤	动作	验收点
1. 预演	测试环境用历史/回放数据验证新版本	结果与基准一致
2. 记录	锁定旧 Jar、配置、JobID、当前并行度	可追溯
3. Savepoint	stop-with-savepoint	Savepoint 成功且路径可访问
4. 新版本	从 Savepoint 启动，显式设置兼容 UID	状态能恢复
5. 观察	Lag、Backpressure、Checkpoint、业务结果	无持续积压/漏数
6. 回滚预案	保留旧 Jar 与 Savepoint	新版本异常能快速退回

注意：如果你改变了 Key 的语义或 State 的数据结构，不能只凭“能从 Savepoint 启动”判断升级安全。业务状态含义是否仍兼容，必须人工设计和验证。

11. 上线前最低生产检查单

上线前：最低生产闭环检查单

正确性	容错	性能	运维
☐ Event Time/Watermark	☐ Checkpoint	☐ 并行度	☐ 监控告警
☐ State/TTL	☐ HA	☐ 反压	☐ 日志
☐ Sink 语义	☐ Savepoint	☐ State 大小	☐ 升级回滚
☐ 手工基准结果	☐ 重启策略	☐ Sink 吞吐	☐ 值班手册

上线不是“部署成功”，而是：能验证、能观察、能恢复、能升级、能回滚。

类别	必须确认
正确性	乱序、迟到、重复、空值、状态恢复后结果均验证
容错	Checkpoint 连续成功；TM/JM 故障演练至少做过一次
HA	外部 HA 存储可用；集群组件故障不会丢失恢复信息
升级	Savepoint、稳定 UID、版本记录、回滚路径明确
性能	目标负载下无持续 Lag；无长期 HIGH Backpressure
监控	业务指标 + Flink 指标 + 外部 Source/Sink 指标都有告警
运维	Job/配置/Jar/Savepoint/日志都可追溯
安全	凭证、ACL、网络和敏感配置不直接写死在代码中

最低闭环：能跑 ≠ 能上线。能上线至少意味着：算得对、看得见、坏了能恢复、升级能回退。

12. 常见误区

误区	为什么错
“有 Checkpoint 就不需要 HA”	JobManager 协调组件自身仍需要故障接管机制
“Savepoint 就是手工 Checkpoint，没区别”	用途和生命周期不同：Savepoint 面向运维/升级
“升级只换 Jar 就行”	有状态作业还涉及 UID、State schema 和拓扑兼容性
“Application Mode 一定优于 Session Mode”	隔离更强，但是否适合取决于组织资源模型
“Kubernetes 比 YARN 新，所以一定更好”	基础设施选择取决于团队已有平台和运维能力
“Job RUNNING 说明生产正常”	可能已经积压、结果错误或 Checkpoint 持续失败
“失败就多重试几次”	永久性错误会变成重启风暴
“资源越大越稳定”	热点 Key、慢 Sink、错误状态设计不会被单纯堆资源解决

13. 练习题

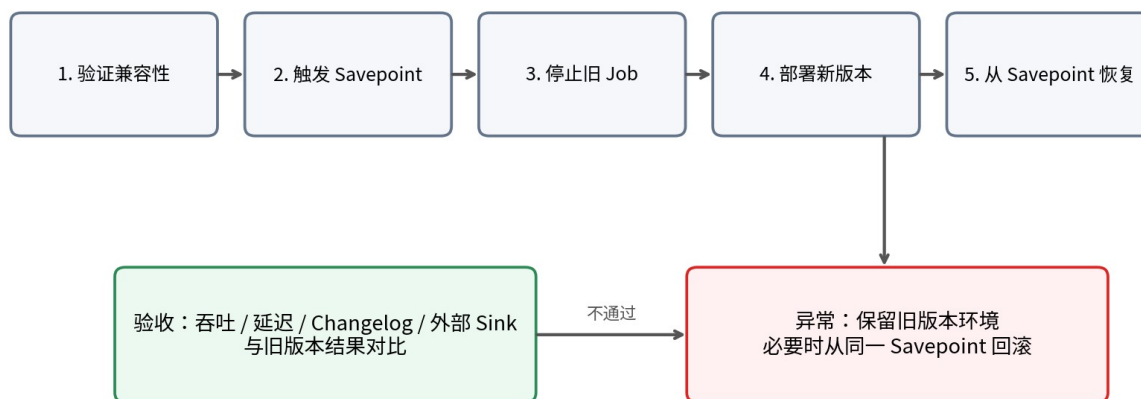
- 1) 为什么 JobManager HA 与 Checkpoint 是两个不同层次的问题？
- 2) 有状态作业升级时，为什么建议先 Savepoint，而不是直接 cancel 再起新 Jar？
- 3) Kafka Lag 上升且 Source backpressure 很高，你为什么不能立即断定 Source 太慢？
- 4) Application Mode 相比 Session Mode，最核心的生产价值是什么？
- 5) 如果新版本从 Savepoint 启动成功，是否就能证明业务升级正确？为什么？

参考答案

- 1) HA 解决协调组件失效接管；Checkpoint 解决作业状态和输入位置的恢复。
- 2) Savepoint 给出明确、可保留的一致状态边界，便于迁移和回滚。
- 3) Backpressure 会从慢下游向上游传播，Source 可能只是被迫减速。
- 4) 一应用一集群带来更清晰的资源/故障隔离和生命周期管理。
- 5) 不能。还要验证状态语义、业务结果、重复/漏数、外部 Sink 语义和性能。

14. 本讲总结

生产升级的安全闭环



把这一讲压缩成一句话：生产 Flink 不是“让 Job 一直跑”，而是建立一套让它在故障、升级、资源变化和外部系统抖动时仍然可验证、可恢复、可观察、可回滚的系统。

下一讲建议：Flink 第 16 讲可以做“Flink 全课程综合大作业”：给出一个完整实时风控/订单系统需求，让你自己完成架构设计、SQL/DataStream 选择、State/Timer、Checkpoint、HA、容量估算和上线检查。

官方参考（截至 2026-09）

- Apache Flink Deployment Overview - stable docs
- Apache Flink Native Kubernetes - stable docs
- Apache Flink Kubernetes HA Services - stable docs
- Apache Flink Operations Playground: Upgrading & Rescaling a Job
- Apache Flink 2.3 Upgrading Applications and Flink Versions